// Security Assessment

12.11.2025 - 12.15.2025

OneSig- Solana

LayerZero

HALBORN

OneSig- Solana - LayerZero

Prepared by: HALBORN

Last Updated 12/24/2025

Date of Engagement: December 11th, 2025 - December 15th, 2025

Summary

100% OF ALL REPORTED FINDINGS HAVE BEEN ADDRESSED

ALL FINDINGS	CRITICAL	HIGH	MEDIUM	LOW	INFORMATIONAL
2	0	0	0	0	2

TABLE OF CONTENTS

1. Introduction
2. Assessment summary
3. Test approach and methodology
4. Risk methodology
5. Scope
6. Assessment summary & findings overview
7. Findings & Tech Details
   - 7.1 Risk of denial of service due to one-step authority transfer implementation
   - 7.2 Possibility of unhandeld index out of bounds errors
8. Automated Testing

1. Introduction

`LayerZero team` engaged `Halborn` to conduct a security assessment of their **OneSig Solana program** beginning on December 11 2025, and ending on December 15, 2025. The security assessment was scoped to the Solana Programs provided in onesig-internal GitHub repository. Commit hashes and further details can be found in the Scope section of this report.

The `OneSig` program is a threshold-based multi-signature wallet implementation. The project enables permissionless execution of pre-approved transactions by M-of-N secp256k1 (Ethereum-compatible) signatures. The program also supports **EIP-712 typed structured data signing** for cross-chain compatibility and uses **Merkle tree proofs** to efficiently batch-authorize multiple transactions.

2. Assessment Summary

`Halborn` was provided 3 days for the engagement and assigned one full-time security engineer to review the security of the Solana Programs in scope. The engineer is a blockchain and smart contract security expert with advanced smart contract hacking skills, and deep knowledge of multiple blockchain protocols.

The purpose of the assessment is to:

- Identify potential security issues within the Solana Programs.
- Ensure that smart contract functionality operates as intended.

In summary, `Halborn` identified some improvements to reduce the likelihood and impact of risks, which have been partially addressed by the `LayerZero team`:

- `Consider implementing two-step authority transfer process.`
- `Explicitly handle index out of bounds errors.`

3. Test Approach And Methodology

Halborn performed a combination of a manual review of the source code and automated security testing to balance efficiency, timeliness, practicality, and accuracy in regard to the scope of the program assessment. While manual testing is recommended to uncover flaws in business logic, processes, and implementation; automated testing techniques help enhance coverage of programs and can quickly identify items that do not follow security best practices.

The following phases and associated tools were used throughout the term of the assessment:

- Research into the architecture, purpose, and use of the platform.
- Manual program source code review to identify business logic issues.
- Mapping out possible attack vectors
- Thorough assessment of safety and usage of critical Rust variables and functions in scope that could lead to arithmetic vulnerabilities.
- Scanning dependencies for known vulnerabilities (`cargo audit`).
- Local runtime testing (`anchor test`)

4. RISK METHODOLOGY

Every vulnerability and issue observed by Halborn is ranked based on **two sets** of **Metrics** and a **Severity Coefficient**. This system is inspired by the industry standard Common Vulnerability Scoring System.

The two **Metric sets** are: **Exploitability** and **Impact**. **Exploitability** captures the ease and technical means by which vulnerabilities can be exploited and **Impact** describes the consequences of a successful exploit.

The **Severity Coefficients** is designed to further refine the accuracy of the ranking with two factors: **Reversibility** and **Scope**. These capture the impact of the vulnerability on the environment as well as the number of users and smart contracts affected.

The final score is a value between 0-10 rounded up to 1 decimal place and 10 corresponding to the highest security risk. This provides an objective and accurate rating of the severity of security vulnerabilities in smart contracts.

The system is designed to assist in identifying and prioritizing vulnerabilities based on their level of risk to address the most critical issues in a timely manner.

4.1 EXPLOITABILITY

ATTACK ORIGIN (AO):

Captures whether the attack requires compromising a specific account.

ATTACK COST (AC):

Captures the cost of exploiting the vulnerability incurred by the attacker relative to sending a single transaction on the relevant blockchain. Includes but is not limited to financial and computational cost.

ATTACK COMPLEXITY (AX):

Describes the conditions beyond the attacker's control that must exist in order to exploit the vulnerability. Includes but is not limited to macro situation, available third-party liquidity and regulatory challenges.

METRICS:

EXPLOITABILITY METRIC (M_E)	METRIC VALUE	NUMERICAL VALUE
Attack Origin (AO)	Arbitrary (AO:A) Specific (AO:S)	1 0.2

EXPLOITABILITY METRIC (M_E)	METRIC VALUE	NUMERICAL VALUE
Attack Cost (AC)	Low (AC:L) Medium (AC:M) High (AC:H)	1 0.67 0.33
Attack Complexity (AX)	Low (AX:L) Medium (AX:M) High (AX:H)	1 0.67 0.33

Exploitability E is calculated using the following formula:

$$E = \prod m_e$$

4.2 IMPACT

CONFIDENTIALITY (C):

Measures the impact to the confidentiality of the information resources managed by the contract due to a successfully exploited vulnerability. Confidentiality refers to limiting access to authorized users only.

INTEGRITY (I):

Measures the impact to integrity of a successfully exploited vulnerability. Integrity refers to the trustworthiness and veracity of data stored and/or processed on-chain. Integrity impact directly affecting Deposit or Yield records is excluded.

AVAILABILITY (A):

Measures the impact to the availability of the impacted component resulting from a successfully exploited vulnerability. This metric refers to smart contract features and functionality, not state. Availability impact directly affecting Deposit or Yield is excluded.

DEPOSIT (D):

Measures the impact to the deposits made to the contract by either users or owners.

YIELD (Y):

Measures the impact to the yield generated by the contract for either users or owners.

METRICS:

IMPACT METRIC (M_I)	METRIC VALUE	NUMERICAL VALUE
Confidentiality (C)	None (C:N) Low (C:L) Medium (C:M) High (C:H) Critical (C:C)	0 0.25 0.5 0.75 1
Integrity (I)	None (I:N) Low (I:L) Medium (I:M) High (I:H) Critical (I:C)	0 0.25 0.5 0.75 1
Availability (A)	None (A:N) Low (A:L) Medium (A:M) High (A:H) Critical (A:C)	0 0.25 0.5 0.75 1
Deposit (D)	None (D:N) Low (D:L) Medium (D:M) High (D:H) Critical (D:C)	0 0.25 0.5 0.75 1
Yield (Y)	None (Y:N) Low (Y:L) Medium (Y:M) High (Y:H) Critical (Y:C)	0 0.25 0.5 0.75 1

Impact I is calculated using the following formula:

$$I = max(m_I) + \frac{\sum m_I - max(m_I)}{4}$$

4.3 SEVERITY COEFFICIENT

REVERSIBILITY (R):

Describes the share of the exploited vulnerability effects that can be reversed. For upgradeable contracts, assume the contract private key is available.

SCOPE (S):

Captures whether a vulnerability in one vulnerable contract impacts resources in other contracts.

METRICS:

SEVERITY COEFFICIENT (C)	COEFFICIENT VALUE	NUMERICAL VALUE
Reversibility (r)	None (R:N) Partial (R:P) Full (R:F)	1 0.5 0.25
Scope (s)	Changed (S:C) Unchanged (S:U)	1.25 1

Severity Coefficient C is obtained by the following product:

$$C = rs$$

The Vulnerability Severity Score S is obtained by:

$$S = min(10, EIC * 10)$$

The score is rounded up to 1 decimal places.

SEVERITY	SCORE VALUE RANGE
Critical	9 - 10
High	7 - 8.9
Medium	4.5 - 6.9
Low	2 - 4.4

SEVERITY	SCORE VALUE RANGE
Informational	0 - 1.9

5. SCOPE

REPOSITORY

(a) Repository: onesig-internal

(b) Assessed Commit ID: cc4c1c1

(c) Items in scope:

- ./programs/onesig/Cargo.toml
- ./programs/onesig/Xargo.toml
- ./programs/onesig/src/instructions/execute_transaction.rs
- ./programs/onesig/src/instructions/close_merkle_root.rs
- ./programs/onesig/src/instructions/set_config.rs
- ./programs/onesig/src/instructions/mod.rs
- ./programs/onesig/src/instructions/verify_merkle_root.rs
- ./programs/onesig/src/instructions/init_one_sig.rs
- ./programs/onesig/src/types.rs
- ./programs/onesig/src/events.rs
- ./programs/onesig/src/constants.rs
- ./programs/onesig/src/lib.rs
- ./programs/onesig/src/state.rs
- ./programs/onesig/src/errors.rs
- ./programs/onesig/src/validation/signature_tests.rs
- ./programs/onesig/src/validation/merkle.rs
- ./programs/onesig/src/validation/mod.rs
- ./programs/onesig/src/validation/signature.rs
- ./programs/onesig/src/validation/merkle_tests.rs

Out-of-Scope: Third party dependencies and economic attacks.

REMEDIATION COMMIT ID:

- https://github.com/LayerZero-Labs/onesig-internal/pull/348

Out-of-Scope: New features/implementations after the remediation commit IDs.

6. ASSESSMENT SUMMARY & FINDINGS OVERVIEW

CRITICAL	HIGH	MEDIUM	LOW	INFORMATIONAL
0	0	0	0	2

SECURITY ANALYSIS	RISK LEVEL	REMEDIATION DATE
RISK OF DENIAL OF SERVICE DUE TO ONE-STEP AUTHORITY TRANSFER IMPLEMENTATION	INFORMATIONAL	ACKNOWLEDGED - 12/22/2025
POSSIBILITY OF UNHANDELD INDEX OUT OF BOUNDS ERRORS	INFORMATIONAL	SOLVED - 12/16/2025

7. FINDINGS & TECH DETAILS

7.1 RISK OF DENIAL OF SERVICE DUE TO ONE-STEP AUTHORITY TRANSFER IMPLEMENTATION

// INFORMATIONAL

Description

The program allows configuring authorized signers and, optionally, a required executor who must sign transactions after they have been approved by the signers. This configuration can be set during program initialization or updated later via the `set_config` instruction.

However, because the program does not implement a two-step authority transfer mechanism, it is possible to misconfigure the signers or executor in a way that prevents future transactions from being signed, resulting in a denial of service.

Specifically:

- Invalid signer accounts can be added, the signature threshold can be increased, or previously valid signers can be removed, leaving no valid set of signers capable of authorizing transactions.
- The `executor_required` flag can be enabled while assigning an invalid executor account that cannot provide a signature.

In both cases, all transactions will be rejected due to missing required signatures. At the same time, it becomes impossible to update the program configuration without redeploying the program, effectively causing a permanent denial of service.

packages/onesig-solana/programs/onesig/src/state.rs

Copy Code

```
13 ruct OneSigState {
14     pub one_sig_id: u64,
15     pub seed: Hash,
16     // The bump for the one_sig_signer PDA
17     pub bump: u8,
18     // Transaction replay protection counter
19     pub nonce: u64,
20     pub multisig: Multisig,
21     pub executors: Executors,
22 }
23
24 #[derive(InitSpace, Clone, AnchorSerialize, AnchorDeserialize)]
25 pub struct Multisig {
26     #[max_len(SIGNERS_MAX_LEN)]
27     pub signers: Vec<Address>,
28     pub threshold: u8,
29 }
30
31 impl Multisig {
32     pub fn add_signer(&mut self, signer: Address) -> Result<()> {
```

```
        require!(self.signers.len() < SIGNERS_MAX_LEN, OneSigError::InvalidSignersLen);
        require!(!self.signers.contains(&signer), OneSigError::DuplicateSigners);
        self.signers.push(signer);
        Ok(())
    }
```

Copy Code

```
#[derive(InitSpace, Clone, AnchorSerialize, AnchorDeserialize)]
pub struct Executors {
    #[max_len(EXECUTORS_MAX_LEN)]
    pub executors: Vec<Pubkey>,
    pub executor_required: bool,
}

impl Executors {
    pub fn add_executor(&mut self, executor: Pubkey) -> Result<()> {
        require!(self.executors.len() < EXECUTORS_MAX_LEN, OneSigError::InvalidExecutorsLen);
        require!(!self.executors.contains(&executor), OneSigError::DuplicateExecutor);
        self.executors.push(executor);
        Ok(())
    }
```

BVSS

AO:S/AC:L/AX:L/R:F/S:U/C:N/A:C/I:N/D:N/Y:N (0.5)

Recommendation

To address this issue, it is recommended to:

- Ensure that at least the required threshold of signers is able to sign transactions after program initialization, before assigning the OneSig PDA public key as an external authority.
- Implement a two-step authority transfer mechanism in which a new signer or executor is first proposed in an initial instruction and must explicitly confirm the role by signing a subsequent instruction. If the new signer or executor does not confirm, the existing authorities remain unchanged.

Remediation Comment

ACKNOWLEDGED: The **LayerZero team** acknowledged this finding , noting that misconfigurations introduced during initialization can be resolved through redeployment with the correct configuration.

7.2 POSSIBILITY OF UNHANDELD INDEX OUT OF BOUNDS ERRORS

// INFORMATIONAL

Description

The `execute_transaction` instruction validates the provided Merkle root, computes the Merkle leaf hash from the supplied transaction data, and executes the transaction if the leaf is confirmed to be included in the Merkle tree.

However, the instruction does not correctly validate the transaction data and the `remaining_accounts`, which may lead to unhandled out-of-bounds errors:

- The `build_instruction` function does not verify that `remaining_accounts` contains at least one account representing a program ID.

packages/onesig-solana/programs/onesig/src/instructions/execute_transaction.rs

Copy Code

```
151 fn build_instruction(
152     one_sig_signer: &AccountInfo,
153     transaction: &OneSigTransaction,
154     remaining_accounts: &[AccountInfo],
155 ) -> OneSigInstruction {
156     OneSigInstruction {
157         program_id: remaining_accounts[0].key(), // The first account is always the program_id
158         accounts: remaining_accounts
159             .iter()
160             .skip(1) // Skip program_id
161             .map(|acc| {
162                 // only the one_sig_signer account can be the signer
163                 OneSigAccountMeta {
164                     pubkey: acc.key(),
165                     is_signer: acc.key() == one_sig_signer.key(),
166                     is_writable: acc.is_writable,
167                 }
168             })
169             .collect(),
170         data: transaction.ix_data.clone(),
171         value: transaction.value,
172     }
173 }
```

- The `execute_instruction` function does not ensure that `remaining_accounts` contains at least two accounts before accessing the slice range `[1..]`.
- The `execute_instruction` function does not validate that the instruction data length is equal to or greater than the instruction discriminator length.

packages/onesig-solana/programs/onesig/src/instructions/execute_transaction.rs

Copy Code

```
180 fn execute_instruction(
181     ctx: &Context<ExecuteTransaction>,
182
```

```rust
    instruction: OneSigInstruction,
) -> Result<()> {
    let balance_before = ctx.accounts.one_sig_signer.lamports();

    // Convert OneSigInstruction to SolanaInstruction
    let (solana_ix, value) = instruction.into();

    // Not allow to re-entry to execute_transaction
    if solana_ix.program_id == ID {
        let discriminator = crate::instruction::ExecuteTransaction::DISCRIMINATOR;
        require!(*discriminator != solana_ix.data[0..discriminator.len()], OneSigError::Reentranc
    }

    // Execute the instruction with the PDA's signature
    invoke_signed(
        &solana_ix,
        &ctx.remaining_accounts[1..], // Skip program_id
        &[&[
            ONE_SIG_SEED,
            ctx.accounts.one_sig_state.key().as_ref(),
            &[ctx.accounts.one_sig_state.bump],
        ]],
    )?;
```

BVSS

AO:S/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N (0.0)

Recommendation

To address this issue, it is recommended to verify that the `remaining_accounts` and the instruction data have valid lengths before accessing them through direct indexing.

Remediation Comment

SOLVED: The **LayerZero team** resolved this finding by verifying that the `remaining_accounts` and the instruction data have valid lengths.

Remediation Hash

https://github.com/LayerZero-Labs/onesig-internal/pull/348

8. AUTOMATED TESTING

Static Analysis Report

Description

Halborn used automated security scanners to assist with detection of well-known security issues and vulnerabilities. Among the tools used was `cargo audit`, a security scanner for vulnerabilities reported to the RustSec Advisory Database. All vulnerabilities published in `https://crates.io` are stored in a repository named The RustSec Advisory Database. `cargo audit` is a human-readable version of the advisory database which performs a scanning on Cargo.lock. Security Detections are only in scope. All vulnerabilities shown here were already disclosed in the above report. However, to better assist the developers maintaining this code, the auditors are including the output with the dependencies tree, and this is included in the cargo audit output to better know the dependencies affected by unmaintained and vulnerable crates.

Cargo Audit Results

The program contains no vulnerabilities detected by `cargo audit`.

Halborn strongly recommends conducting a follow-up assessment of the project either within six months or immediately following any material changes to the codebase, whichever comes first. This approach is crucial for maintaining the project's integrity and addressing potential vulnerabilities introduced by code modifications.